

Software Documentation

General Approach (applied to both open and obstacle challenge)

1. [Odometry](#): Estimating pose using [encoder](#) and [gyro](#)
2. [Localisation](#) using LiDAR using [Iterative Closest Points](#) (previously [Spike Landmark](#)) before merging both the odometry pose and localised pose using a [complementary filter](#)
3. [Path following](#) using the new merged pose

Sensor Libraries

LiDAR - COIN-D4

We created our own library, because there weren't any python raspberrypi libraries, to receive data from the COIN-D4 LiDAR which uses the UART protocol. To create it, we used the serial package to open the serial port `/dev/ttyS0` with the baud rate 230400.

Everytime the input buffer is populated, we check if the first two characters match the data header `'\xAA\x55'` before retrieving the sample count (4th byte) so that every group of readings is fully taken before checking the checksum and parsing it. The scan frequency is roughly 10 rotations per second and each frame of data points is 360 degrees.

Afterwards, each reading is assigned to the closest angle it was taken from, by rounding the angle and taking a modulus of 360 as the index.

Gyro - MPU-6050

The gyro sensor uses the i2c protocol so initially, we used the pigpio library to receive data, but realised it was too slow and caused significant delays, so we switched to smbus.

Additionally, we had to numerical integrate the z-axis gyroscope data over time to calculate the heading:

- Read the raw angular velocity
- Subtract the calibrated gyro offset value
- Calculate the time passed since the previous reading
- Take the sum of the current and previous z readings, multiply it by the time passed, and divide it by the sensitivity scale factor ($131 * 2$)
- To find the calibrated gyro offset value, we also created a calibration function so that when we leave the robot as stationary, we can take the average of 500 measurements (which are supposed to be zero values) to find the z error. We also created functions to save the calibration as a txt file and load it.

Encoder - AB Hall-effect Encoder

It is a type of quadrature encoder that measures rotation using the strength of the magnetic field in the x and y direction, and measures the positive/negative magnetic charge in those directions. The true mechanical angle is then recovered by applying arctangent because the rotating magnet produces orthogonal sine and cosine magnetic field components.

We used the pigpio library and programmed two callback functions to detect change in states for each channel A (Pin 5) and B (Pin 6)

```
cb1 = pi.callback(5, pigpio.RISING_EDGE, self.step_count)
```

```
cb2 = pi.callback(6, pigpio.EITHER_EDGE, self.drive_dir)
```

In the first callback, when channel A rises, it checks the state of channel B. If B is high, it is moving backward, whereas if B is low, it is moving in the opposite direction, forward. The second callback is constantly updating the state of channel B.

Encoder calibration:

2025

1st attempt: 1522.4mm – 5113 steps, 0.298mm – 1 step

2nd attempt: 4670 mm – 15795 steps, 0.296mm – 1 step

2026

Actual	Encoder	Time	Notes
1285	2705	5 s	

2005	4370	8 s	Discarded
2075	4365	8 s	
2070	4357	8 s	

Motor Driver - TA6586

The TA6586 is a monolithic, bidirectional H-bridge motor driver IC that uses sign-magnitude drive. In sign-magnitude control, the sign determines the motor's direction, while the magnitude determines how strongly the motor is driven. In the library, the sign of the speed argument selects the direction of rotation, while the absolute value of speed determines the PWM duty cycle and by extension the motor speed.

For positive speed, the GPIO pins 20, 21 are configured as such

```
self.pi.set_PWM_dutycycle(21, 255-speed) # PWM controlled
self.pi.set_PWM_dutycycle(20, 255) # constantly high
vice versa for negative speed
```

```
self.pi.set_PWM_dutycycle(21, 255) # constantly high
self.pi.set_PWM_dutycycle(20, 255+speed) # PWM controlled
```

This allows for lower switching losses because fewer transistors switch simultaneously

Camera - Raspberry Pi Camera Module v2

We use the Raspberry Pi Camera Module with the picamera2 and cv2 library to identify the colour of the obstacles in front of the cars.

The colour detection works as follows:

1. Remove the top 105 pixels from the image to prevent the camera from seeing over the wall, avoiding any false positives.
2. Apply a gaussian blur on the image to remove noise and small variations in the image.
3. Convert the image from BGR to HSV. HSV is more useful in this case as it separates the hue from the brightness of colours, making it more reliable under different lighting conditions.
4. Using cv2.inRange(), a binary mask is applied to the image, where the white parts of the image represent the areas that are within the tested thresholds for the green and red obstacles

5. Run `cv2.SimpleBlobDetector()` on the binary masks. It rejects blobs that are smaller than 650 pixels to remove any noise

For each colour `detect_blobs()` will only return the largest blob that is detected.

Servos (Camera Swivel, Steering) - MG90

To calibrate the servos, we tested with different values from 500 to 2500 microseconds (μ s) to find the centre, maximum and minimum.

Servo	Centre	Max, Min (in degrees)
Camera	1475	87
Steering	1425	42

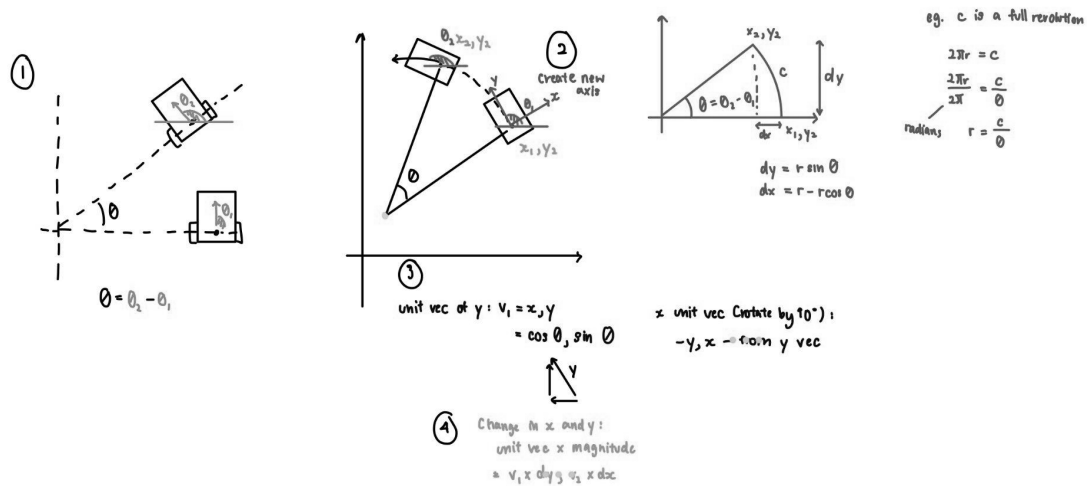
I/O Interface (Button, LEDs)

In total, we have a button to start and stop the programme and three LEDs (red, yellow, green) to signal different messages for different challenges: open (red -- stationary, green -- running), obstacle (yellow -- stationary, green -- sees green obstacle, red -- sees red obstacle).

Odometry

We use the wheel rotation and gyro to estimate the robot's pose (position and heading) while it is running.

- `steer_in_dir()`: steering (proportional control)
- `augment_path()`, `augment_paths()`: Steering back to line
 - Find path vector = $(x_2 - x_1) / (y_2 - y_1)$
 - Find distance of path
 - Find path vector in units
 - Find perpendicular path vector
 - Find path direction
- `dot()`: dot product
 - how far robot is off the path – `drive_path()`
 - how far robot has traveled along the path
- `drive_path()`, `drive_paths()`:
- `estimate_pose()`:
 - First version: looked at the heading and assumed a straight path based on the heading
 - Second version: treat the movement as an arc of a circle



First, we take the centre of rotation as the origin, which means

$$\theta = \theta_2 - \theta_1$$

$$d_y = r \sin \theta, d_x = r \cos \theta$$

$$r = \frac{c}{\theta}$$

in which c is the distance travelled

$$d_y = \frac{c}{\theta} \sin \theta, d_x = \frac{c}{\theta} \cos \theta$$

To apply this back to the original axis, we take the x, y vectors of the "morphed" graph, (rotate y vector by 90°), before we multiply the unit vectors with the local dx, dy to find the global dx, dy

$$y' = (\cos \theta, \sin \theta)$$

$$x' = (-\sin \theta, \cos \theta)$$

$$d_x = x' \times d_x, d_y = y' \times d_y$$

Afterwards, we integrate other localisation methods on top of using odometry, by using a complementary filter.

Localisation

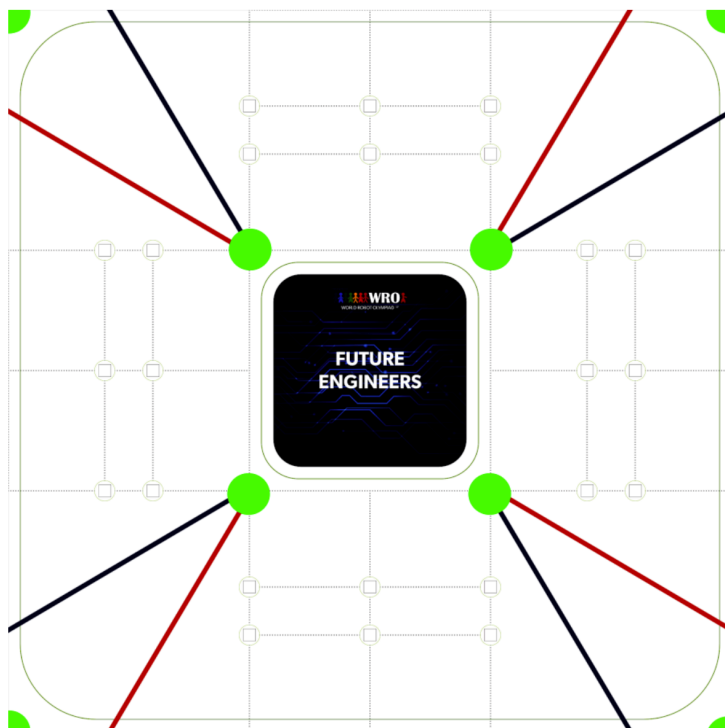
Spike Landmark (Old)

Initially, we used the spike landmark method during the 2025 season of WRO, but we wanted to improve our localisation method so we have since shifted to the [Iterative Closest Point](#) method.

Essentially, the method identifies which lidar readings are spikes or points of interest, in our case corners which are noticeably further away than the adjacent readings.

Afterwards, we calculate if there is any difference between the supposed and actual cartesian coordinate of the landmark to find the error.

- Defined 8 landmarks: 4 inner corners, 4 outer corners, marked as green circles



- `add_cartesian()`: convert lidar readings into cartesian coordinates
- `identify_spikes()`
 - the different between two lidar measurements adjacent to the centre measurement have to show a significant difference
 - filter out spikes within 250mm

- `match_landmarks()`: within 120mm, which we tested as the most optimal threshold
- `calc_position_error()`, `calc_angle_error()`: according to the spikes matched, we calculate the total error accumulated for x, y and heading (angle of the spike from the robot) separately before finding the average for each value and adding it to the `odometry_pose`

However, this method only uses a few points out of the approximate 400 we get from the lidar in comparison to the [Iterative Closest Point](#) method that makes full use of all the lidar readings.

Iterative Closest Point (current)

- `augment_walls()`: Define the 8 walls on the playfield, getting their path vector, path direction, unit path vector and perpendicular path vector which are values needed
- `add_cartesians()`: same as the one used in the Spike Landmark

localise(): Let's start by transforming 1 point on a cartesian plane.

To handle the rotation, we can

$$x_2 = R \cos(\alpha + \beta) = R[\cos(\alpha) \cos(\beta) - \sin(\alpha) \sin(\beta)]$$

$$y_2 = R \sin(\alpha + \beta) = R[\sin(\alpha) \cos(\beta) + \cos(\alpha) \sin(\beta)]$$

Substitute $x_1 = R \cos(\alpha)$, $y_1 = R \sin(\alpha)$,

$$x_2 = x_1 \cos(\beta) - y_1 \sin(\beta) \quad y_2 = y_1 \cos(\beta) + x_1 \sin(\beta)$$

Since the change in x, y and heading is going to be small as the frequency of the Coin-D4 is 10 Hertz, we can linearize the functions, by taking their gradients near 0.

$$\cos(\beta) \approx 1, \sin(\beta) \approx \beta$$

$$\therefore x_2 = x_1 - y_1(\beta), y_2 = y_1 + x_1(\beta)$$

Afterwards, we add translation as T_x, T_y

$$x_2 = x_1 - y_1(\beta) + T_x, y_2 = y_1 + x_1(\beta) + T_y$$

$$\therefore x_2 - x_1 = -y_1(\beta) + T_x, y_2 - y_1 = x_1(\beta) + T_y$$

When we put it in a matrix form,

$$\begin{bmatrix} -y_1 & 1 & 0 \\ x_1 & 0 & 1 \end{bmatrix} \begin{bmatrix} \theta \\ T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix}$$

And we can add as many points as we want to the equation

$$\begin{bmatrix} -y_1 & 1 & 0 \\ x_1 & 0 & 1 \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} \theta \\ T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \\ \vdots \end{bmatrix}$$

To solve this $Ax = B$, we take the pseudoinverse of A multiplied by B, which we calculate using the [numpy](#) library.

Now that we have the transformation matrix, we need to identify which point on the wall does the lidar reading correspond to, in our case we find the shortest perpendicular distance between each wall and the reading.

$$dist = \hat{v} \cdot l,$$

l is the vector of the lidar reading from the start of the wall: if it is a vertical wall, we only solve for the x-value and vice versa for a horizontal wall.

localise_iter(): Since we are dealing with an overdetermined system, there is no exact solution, but only a rough estimate. Running it with more iterations will help minimise the errors.

We tested it with a set of fake data which started with the mini-sets that have far fewer readings, before moving on to set 1 which has 360 points, closer to what we will expect in the actual run. After debugging, we realised that it was able to get close to the actual pose with only one iteration. Additionally, we timed it and it took roughly 1/50th of a second, meaning it is able to keep up with the lidar which takes 10 readings every 1 second.

Complementary Filter

We set very small percentages for the localisation pose because we found that the odometry pose was already quite accurate.

POSITION_FILTER_RATIO = 0.1

HEADING_FILTER_RATIO = 0.01

Camera Tracking

- Plotted points along the playfield, in between every pair of squares, where the obstacle could be placed
- Camera will track two points ahead of its current position
- Reused an existing function (dir_to_point) to calculate the angle the servo needs to turn to the obstacle, but adjusting the range to -180 to 180, as the function is used for cases where the range is 0 to 360

Path following

- drive_path(), drive_paths():
 - Find the error by taking the dot product of the unit vector perpendicular to the path and the vector from the start of the path to the robot (robot_vector):
perpendicular displacement = $\hat{u} \cdot \vec{r}$
 - Get the correction / perpendicular distance of the robot from the path by multiplying the error by a gain
 - Add the target_dir of the path with the correction which is what the robot will steer towards
 - Keep checking the dist_travelled_along_path if it reaches the end by taking the dot product of the unit_path_vector and the robot vector: displacement along path = $\hat{p} \cdot \vec{r}$

Open Strategy

During the initialisation, the robot will assume it is moving clockwise and travel the first path until it reaches the corner: this is where the robot will read the distances from the lidar sensor for the points (0mm, 2700mm) and (1000mm, 2700mm). If the distance for the left point is greater than the right point, it will deduce that it is moving counterclockwise and update its pose accordingly. The reason why we chose these points instead of measuring the left and right distances is because we encountered some issues with determining the sides at certain start positions.

In the rest of the run, the robot will follow a preplanned clockwise and counterclockwise path, which is as close to the centre of the lanes as possible. By default, it would take the outermost path, 50 cm away from the outer wall. However, if the lidar detects that the first section of the inner wall is extended, the lane will shift closer to the wall such that it is 30 cm away from the outer wall.

We plan on improving our strategy by adjusting the paths to the inner paths where possible.

Pseudocode:

find initial position

first path run to confirm pose

main run:

 (process described in [general approach](#))

 if at ending position:

 stop

Obstacle Strategy

The robot will determine its direction (clockwise or counterclockwise) by getting the front, rear and side distances because the initial position is generally fixed. It will check if an obstacle is in front of it before moving back.

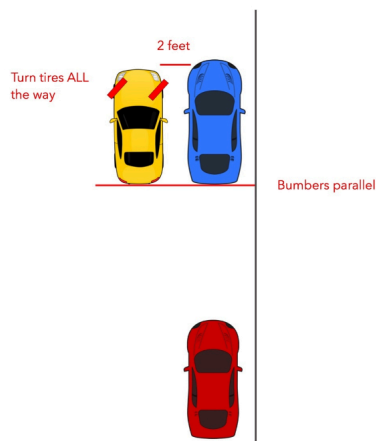
In the main run, the robot will switch between two fixed sets of paths, an outer and inner lane. Depending on the colour of the blocks in front, it will switch between the two paths to follow.

For example, if the robot is moving clockwise, detecting a red block will cause it to follow the inner paths, and vice versa for the green blocks. This will switch if the robot is moving counterclockwise. An improvement we made since 2025 is that we implemented camera tracking such that the camera follows the colour of the obstacle two sections ahead which ensures that the robot never misses the obstacle. In 2025, a major issue we had is that because our robot was larger and the field of vision of the robot was wider, we had to keep readjusting our paths so that the robot would not miss the obstacles, especially at the corners.

As of 25th August, the parking programme is still unfinished, but our strategy is to get into the exact position at the start of parking by moving the robot back and forth. As for the parallel parking procedure, it involves

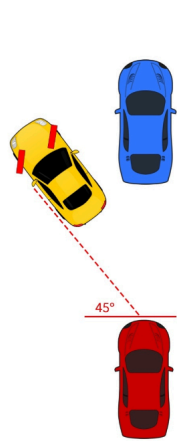
(Scenario in which the parking area is on the right)

- Turn right to 45, move back till vehicle is facing 45
- Turn left to 0, move back till vehicle is behind the barrier
- Turn left to 45, move back till vehicle is facing 0



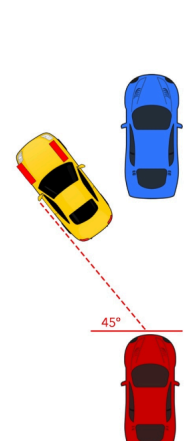
STEP 1

Start two feet from forward car, with your bumpers parallel, and wheels turned All the way around



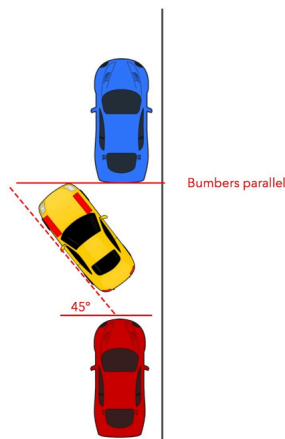
STEP 2

Keep backing up, with wheels fully turned untill you can see the back car's license plate in side mirror. That means you are at 45°



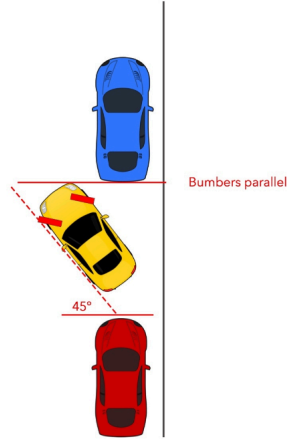
STEP 3

Straighten wheels, and back up



STEP 4

Stop backing up, once the front of your car, is past the bumper of the first car.



STEP 5

While **Stopped**, Turn wheels all the way around



STEP 6

Your parked!

Pseudocode

find initial position

if obstacle in front: move back

main run:

(process described in [general approach](#))

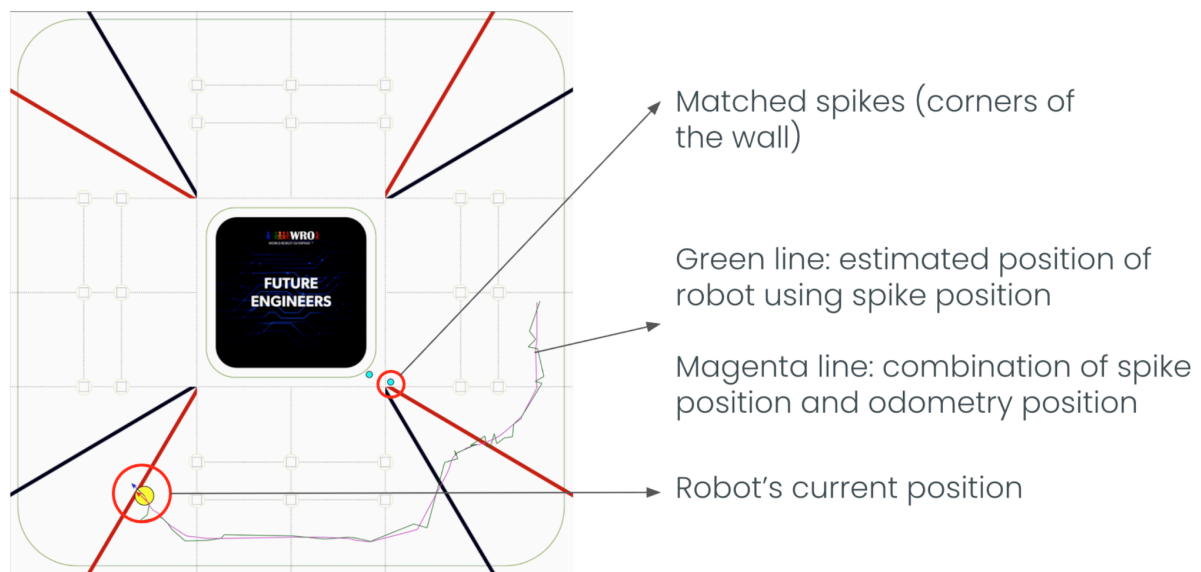
if at ending position:

Parking

Debugging / Telemetry tools

To make it easier to visualise the different variables — robot pose, lidar readings, matched spikes, colour of block detected - we created a telemetry software, using sockets to transfer data from the raspberrypi (client) to a computer (server) with our own protocol. We also keep the messages short so that it all fits in one packet and is not sent too frequently. Afterwards, we used tkinter to draw out the points on the playfield with the exact values written out at the left side of the interface.

Additionally, we changed our initialiser.sh file for easier debugging. Now, every time we run, the programme will log the standard output and error messages to a file with the date and time. To change the mode of the robot (sleep, open, obstacle), we will create a softlink between `main_obstacle/open/sleep.py` to `main.py`.



Hardware Documentation

After a long break from WRO Internationals 2025, we decided to come back to the studio and start work on our next robot for Future Engineers 2026. While our 2025 FE robot was commendable, it had multiple flaws that needed fixing. Upon starting our preparation, we always fix the mechanical issues first, before moving on to improving our code for the robot itself. As such, we came together and formed a list of flaws that our previous robot had.

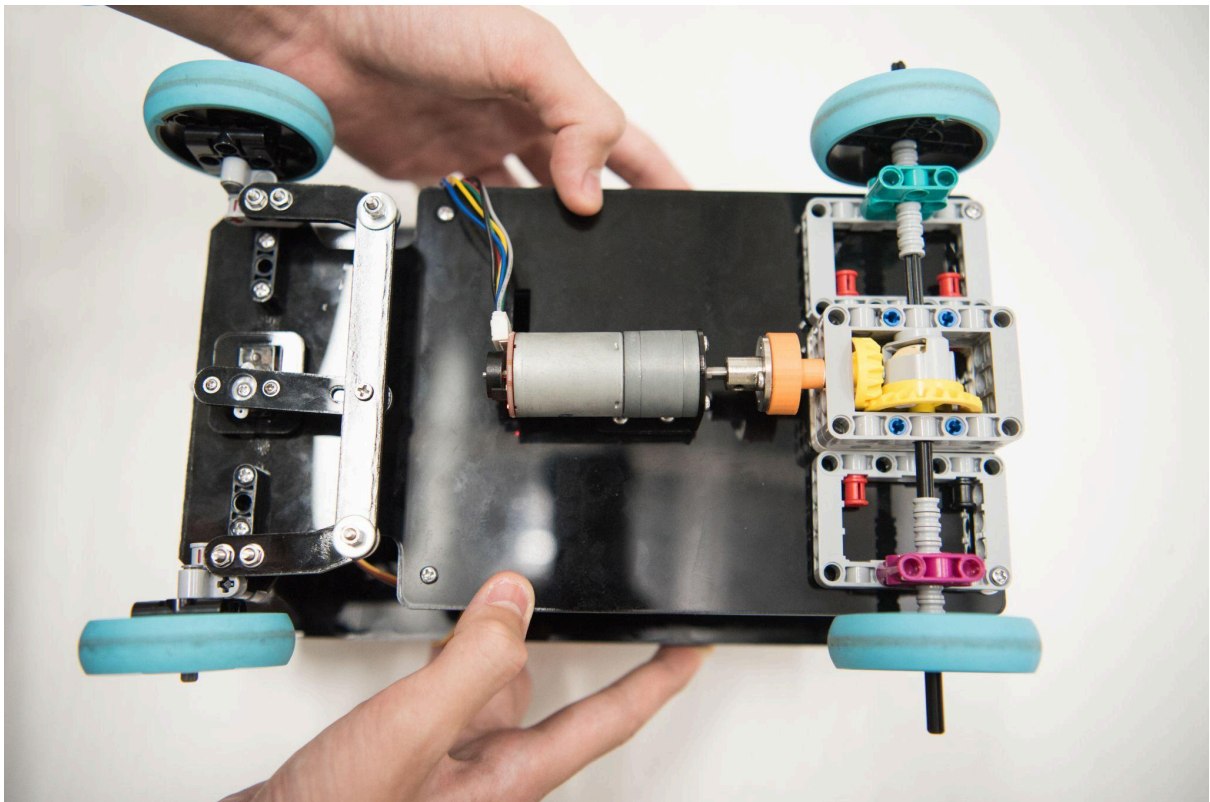
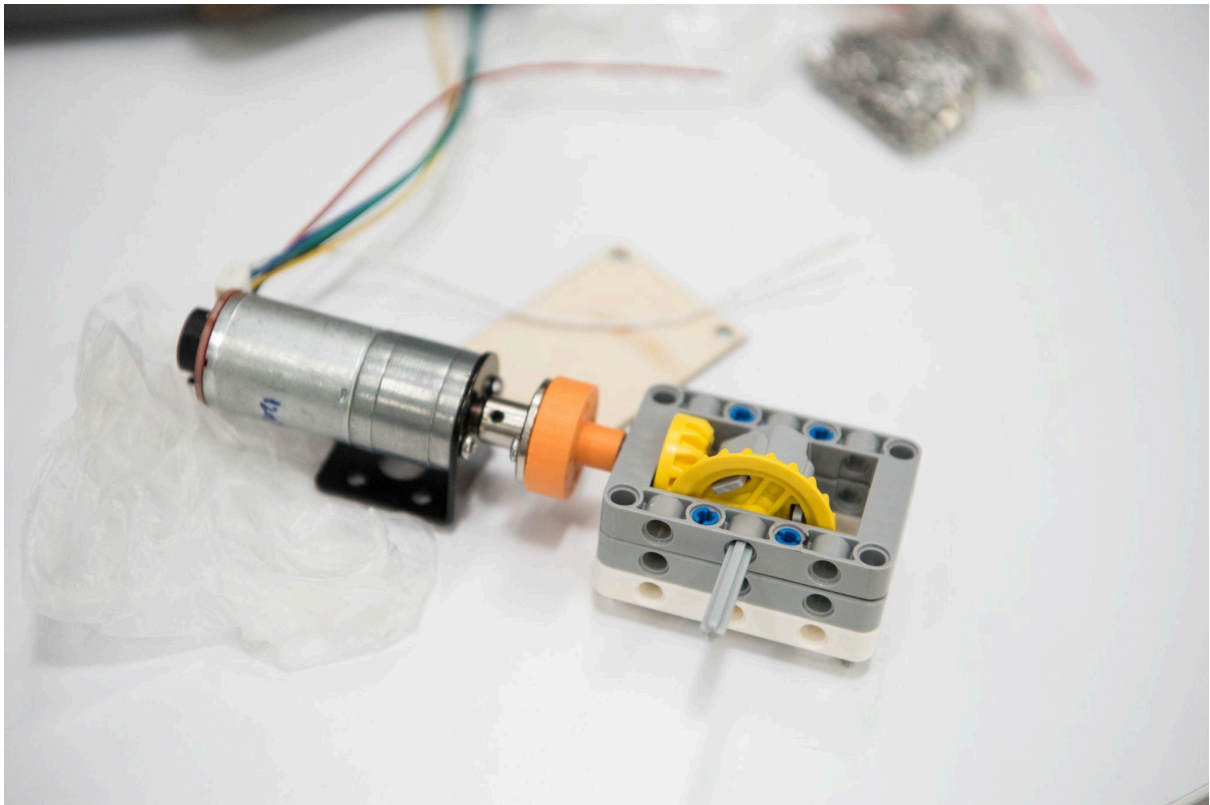
- **Large robot base** - in turn causing a wide turning radius
 - Led to many near-miss instances of the robot and the obstacles during our obstacle challenge runs in the competition
- **Servo motor for our steering mechanism was not centred**
 - To adapt to this issue, we initially cut a slot in our steering component for the parts to have more slack when turning, making it smoother
 - However, this slack in the parts also caused the steering mechanism itself to be quite unsturdy and loose, which could sometimes lead to inaccurate turns in our robot runs
- **Steering wheel components were loose** - attached only by the pivot point, and nowhere else
 - Because of the singular mounting point, the steering wheel segments often started to bend outwards, due to the force being applied downwards on the base because of our electronic components on the top plate of the robot
 - As the steering wheels bend outwards, steering starts to become less smooth, and turning radius is also slightly affected
 - Even worse, our steering wheels would sometimes come out due to this looseness issue
 - However, this looseness did provide a small amount of suspension for our robot, which allowed it to have more grip on uneven surfaces (at times when the field can get slightly offset)
- **Camera was fixed in one direction** - entire robot had to turn to allow the camera to see obstacles on the far end of its POV
 - Despite using a wide angle lens for our camera last year, it was often not wide enough to detect obstacles at the far end, which may sometimes not appear in the camera's POV
 - Results in making incorrect movements to avoid the obstacles, losing points for our competition runs

After forming this list of main issues, we decided to tackle these issues one by one, throughout a few studio sessions.

Reducing robot base size

The first issue we fixed was the **large robot base**. This problem arguably had the most significant impact on our performance, as it was the lead contributor to our failed runs last year. Because of this significance, this issue also came with major changes that had to be done to our robot. This is because simply shrinking our robot frame was not as easy as it seemed; we would need to make our drive base + our Ackermann Steering segment significantly more compact, as well as shrink the layout of our electronic components on the top plate. This process would last a few sessions, but we mainly focused on shrinking the drive base this session.

Initially, our drive base motor, driving the entire RWD module, was mounted perpendicular to the RWD module itself.

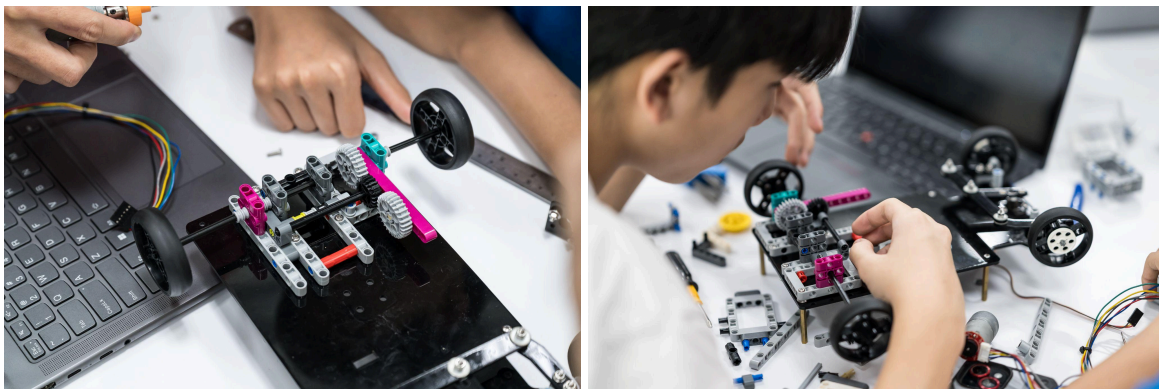


As seen above, this was the most convenient and direct way to mount our drivebase motor at that time. However, the DC motor we are using was decently long, and by mounting it perpendicularly this way, it took up more than $\frac{1}{3}$ of our previous robot's total length. We

realised that if we wanted to make our robot smaller and more compact, we would need to mount this DC motor in another way.

As such, we started to take apart our previous RWD module, and start from scratch again.

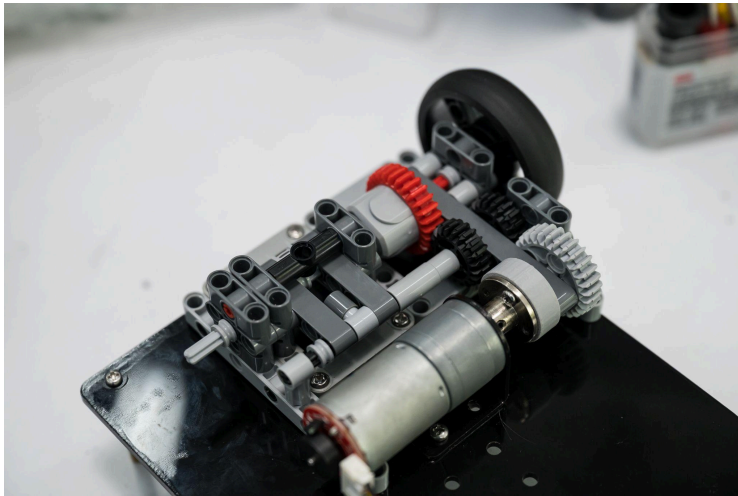
After significantly reducing the size of the RWD module, we still had to similarly reduce the size of our Ackermann Steering module in the front of the base plate, so that we could then reduce the size of the entire base plate. Initially, our steering module was not designed to be compact, as it was our first ever iteration. However, in last year's competition, because of how wide our turning radius was, the robot often struggled to avoid the obstacles during its runs. The solution to this came in two parts; one was to shorten the length of our robot, reducing the distance between our front and back wheels; and the other was to reduce the distance between our steering wheels, to make our steering angle sharper.



Over multiple prototypes and a tedious process of trial and error, we eventually built a suitable RWD module that was much more compact than the previous iteration. This was mainly achieved by mounting our DC motor parallel to the RWD module, instead of perpendicular. This cut approximately 8-9cm from the total length of our robot base, a significant cut from our previous robot.

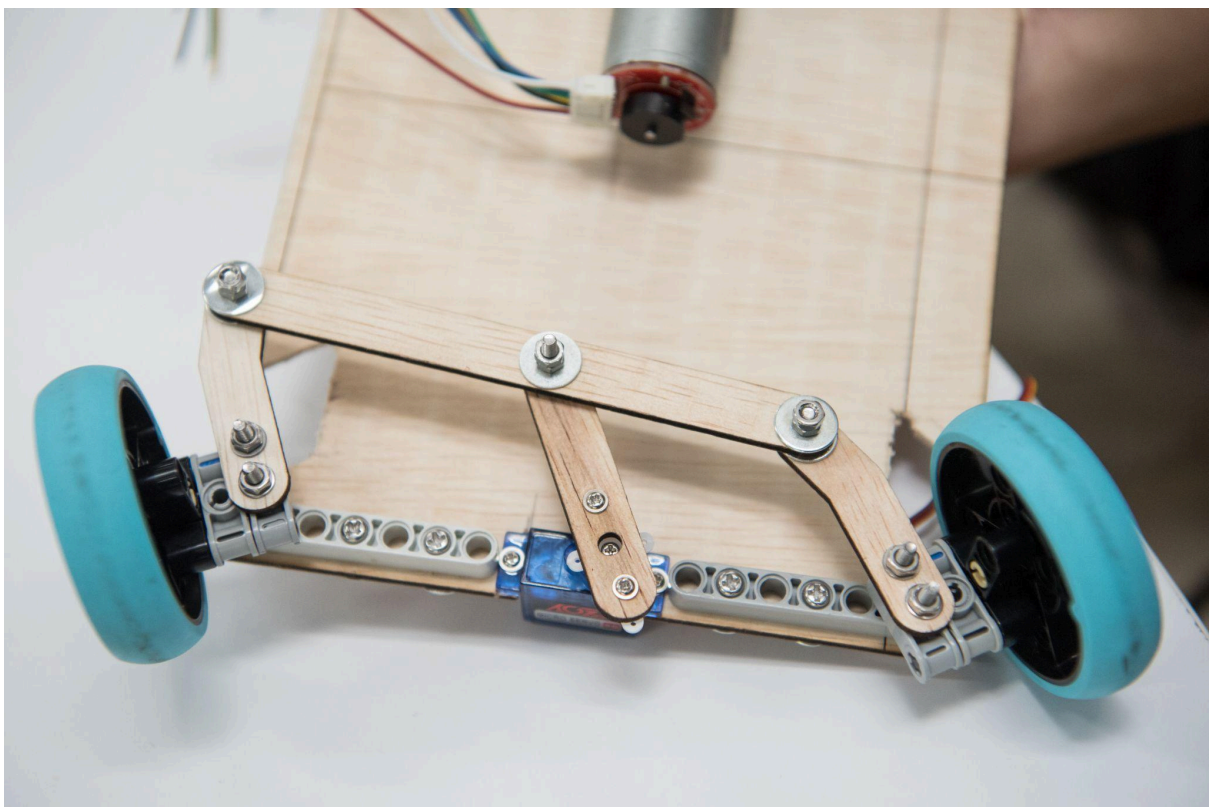
However, mounting the DC motor in a parallel orientation came with a few issues, mainly because the motor would either be too close or too far to the rotating differential gear and shaft. The solution that we eventually found was to gear the DC motor shaft to our main RWD module shaft. This ensured our motor was mounted in an optimal position, while also slightly increasing the speed of our drive base due to the gear ratio of the shafts.

In addition, there was a lot of useless space taken up by our wheel shafts that could be cut from both sides to reduce the width of our robot. After all these changes, our new RWD module looked like this:



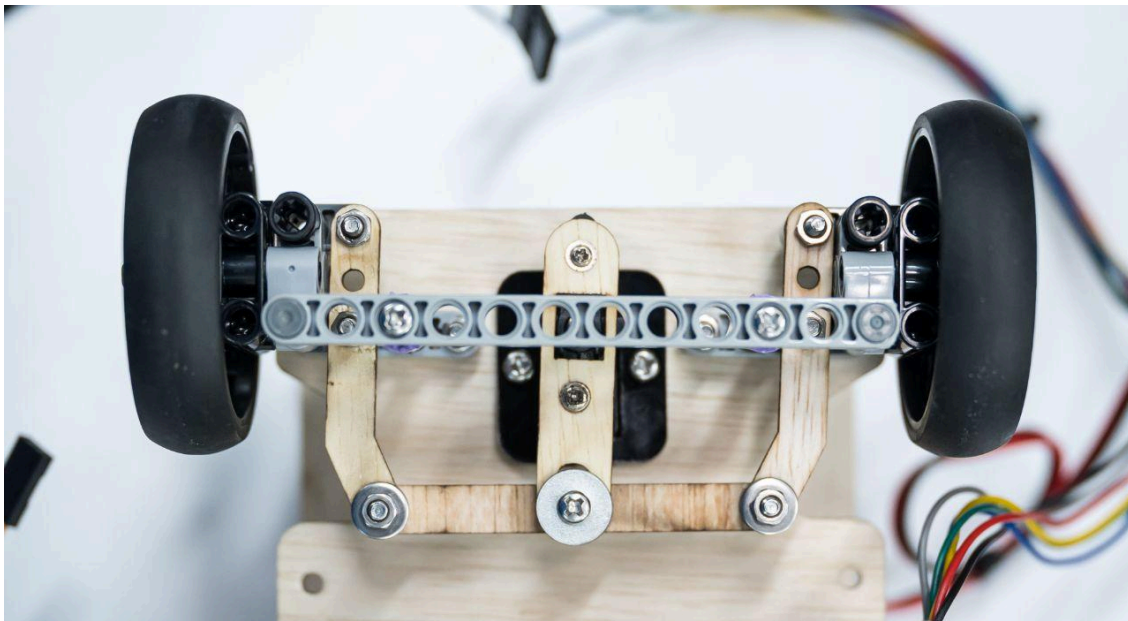
Ultimately, we not only cut 8cm in length, but also 4-5cm in width.

After significantly reducing the size of the RWD module, we still had to similarly reduce the size of our **Ackermann Steering module** in the front of the base plate, so that we could then reduce the size of the entire base plate. Initially, our steering module was not designed to be compact, as it was our first ever iteration.



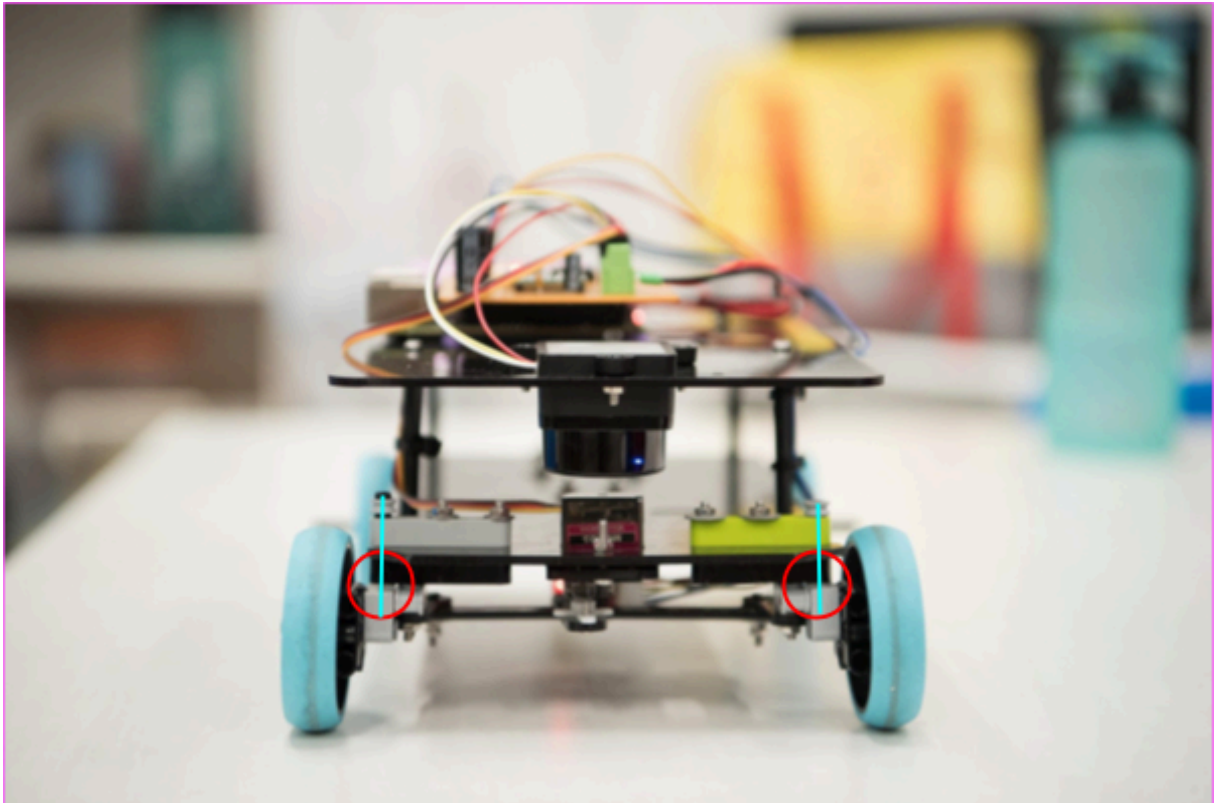
(^^ first steering mech iteration)

However, in last year's competition, because of how wide our turning radius was, the robot often struggled to avoid the obstacles during its runs. The solution to this came in two parts; one was to shorten the length of our robot, reducing the distance between our front and back wheels; and the other was to reduce the distance between our steering wheels, to make our steering angle sharper. As seen in the image above, the distance between both our steering wheels could be reduced by quite a substantial amount, allowing our robot's width to be reduced as a whole. This fix was pretty straightforward and simple, and after a short period of time, we were able to reduce the size of the steering module.



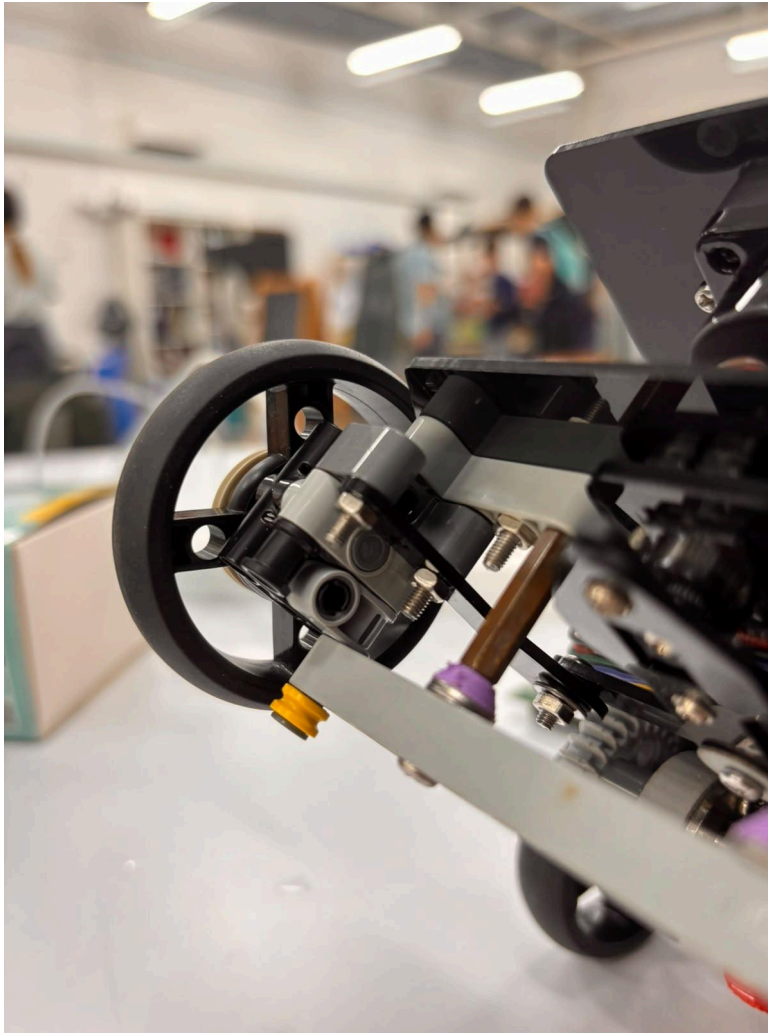
Comparing both iterations of our Ackermann Steering mechanism, we can see that the distance between both wheels has been shortened from ~18 Lego Technic holes to ~13 Lego Technic holes, which is equivalent to ~40mm, or 4cm. Because of this change in width, the length of our Ackermann steering liftarms also had to be edited accordingly to account for this difference. By designing the liftarms online, this process was very fast and we managed to cut out the new liftarms with plywood and connect them quickly.

In our previous robot, the shaft connecting our wheel components to our main robot base plate was short, and only supported by a height of two lego beams. This meant that the shaft still had leeway to bend slightly.

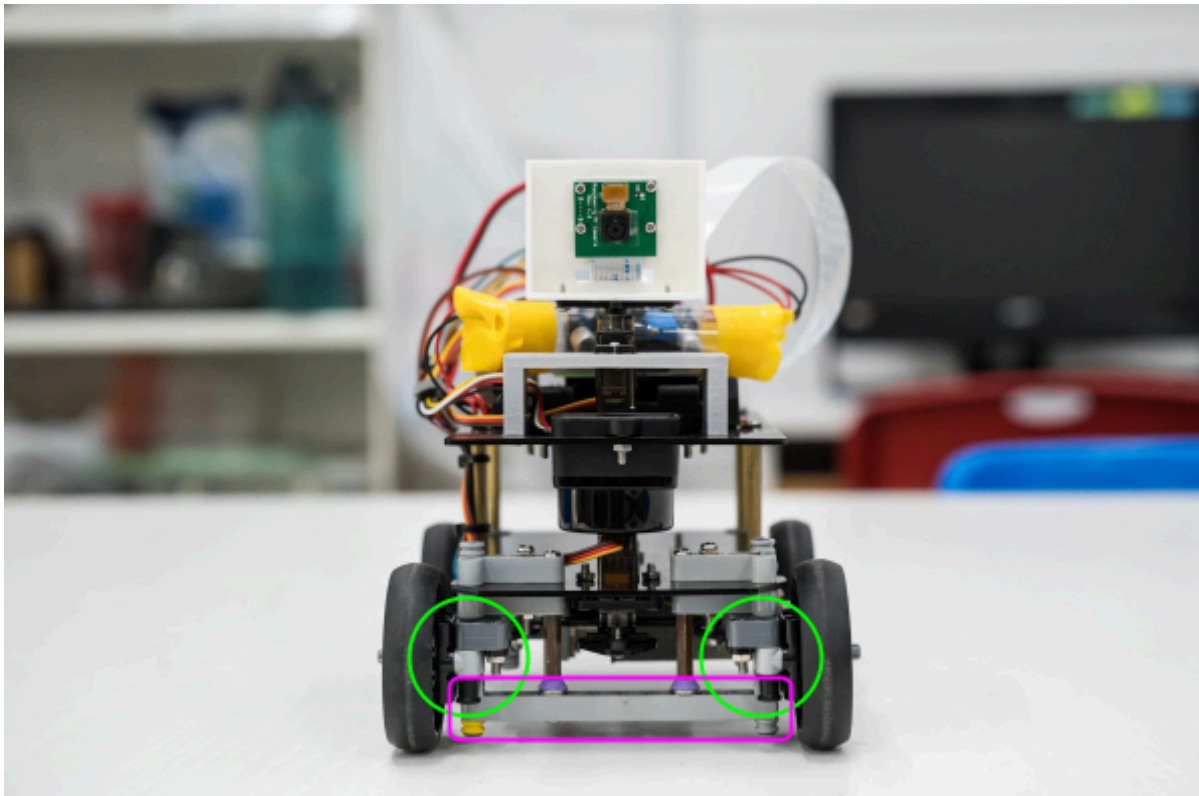


As seen above, our previous robot is in a stationary state, and the wheels are already bending outwards slightly. The connection point, indicated by the red circle, is the only connection point for each wheel.

Fix #1: We redesigned the component linking the shaft, Ackermann liftarm and rotating wheel together, so that the shaft would be supported by 5 Lego beams compared to 2.



Fix #2: Instead of the wheel components having only one connection point on the top, we added a liftarm below the wheel components, linking both wheels together. This beam pulls both wheel shafts together, ensuring both beams remain parallel to each other, and remain perpendicular to the ground.



As indicated by the green circles, it shows fix #1, our new and improved wheel module, and the pink rectangle indicates fix #2, the beam that braces both wheels together. In addition, we braced the beam to the main robot baseplate using two short standoffs, which doubled up as a limiter for our Ackermann Steering linkage mechanism, preventing this mechanism from “overlocking”. Overlocking occurs when both Ackermann liftarms form a straight line with the pivot point, and when the servo motor tries to turn back, the force is exerted straight through the joints instead of turning them, since there is zero leverage to start the rotation. By using these standoffs, it prevents the liftarms from aligning to a straight line, and maintains leverage for continuous rotation and steering.

Centering Steering Servo

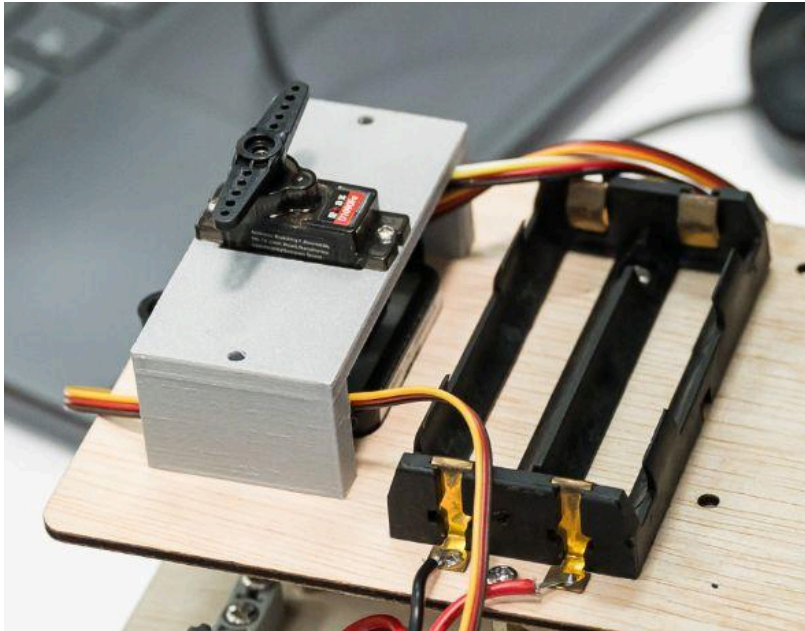
Another issue with our previous robot was the slight inaccuracy in design in our first iteration of our Ackermann Steering module. In our previous robot, we mounted the steering servo motor, by centering the rectangular shape of the servo on the robot base plate. However, this was an issue, as the actual point of rotation is slightly offset from the centre of the rectangular servo motor. As a result, we had to cut a long slot on our Ackermann liftarm to account for this slight inaccuracy, in turn making our steering looser than usual.



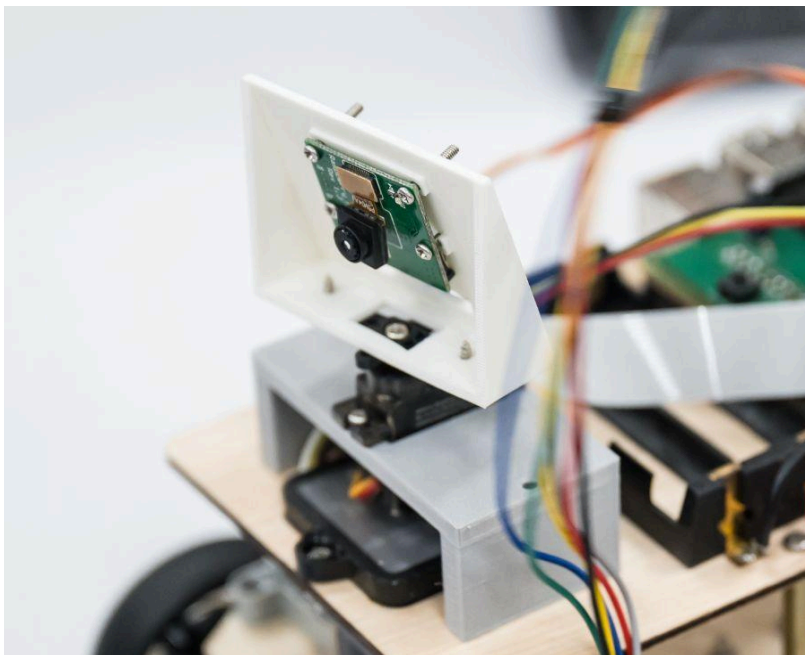
To fix this issue, we had two options. The first option was to mount this servo motor in the same orientation, but offset it slightly to the left to account for the offset of the centre of rotation. The second option was to rotate this servo motor 90 degrees, so the short side of the motor faces the front of the robot. In this way, the centre of rotation does remain centred, while the entire servo motor is also centred to the robot. The former option, however, did not work after testing, as the servo motor would hit the Ackermann Steering bracing on either side of the robot, and thus would not fit if the orientation was kept the same. As such, we decided to go with the latter option. This fix was simple enough, as we just needed to rotate the rectangular slot for the servo motor on our robot base plate CAD file. Soon after, we adjusted our Ackermann steering liftarms accordingly, and our Ackermann Steering module was complete.

Adding Camera Servo

The last main hardware issue we found in our previous robot was the intermittent blindspots of our camera. Despite using a wide-angle lens for our camera, our robot always tends to miss obstacles at the very edge of the Camera's POV, especially at junctions requiring sharp turns. As such, we decided to make a significant mechanical change to how our camera was mounted. Instead of just a fixed and stationery mount, we decided to attach our camera to a servo motor, with a functionality similar to that of a Camera Gimbal module. To achieve this functionality, we first had to find a way to mount the servo motor on the top plate of our robot, while not interfering with the other electronic components.

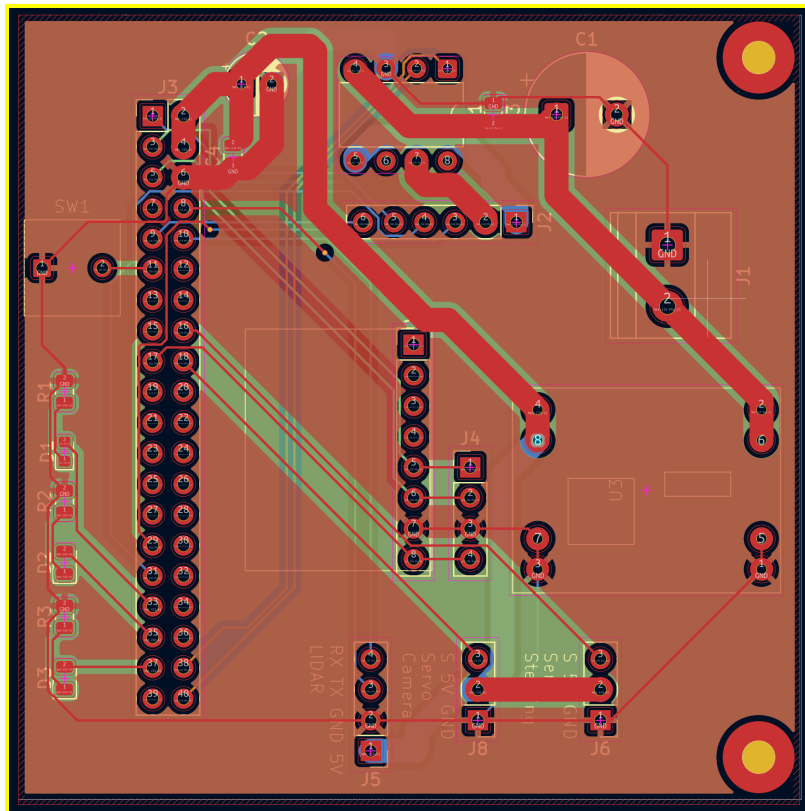


As seen above, the most convenient location we found for our servo motor was to mount it on top of our lidar sensor. This was achieved by designing a custom 3D printed elevated bracket, fastened to the top plate, that allows our servo motor to fit perfectly on top. Thereafter, we quickly attached the camera bracket together with the camera itself, and the rotating camera module was complete.

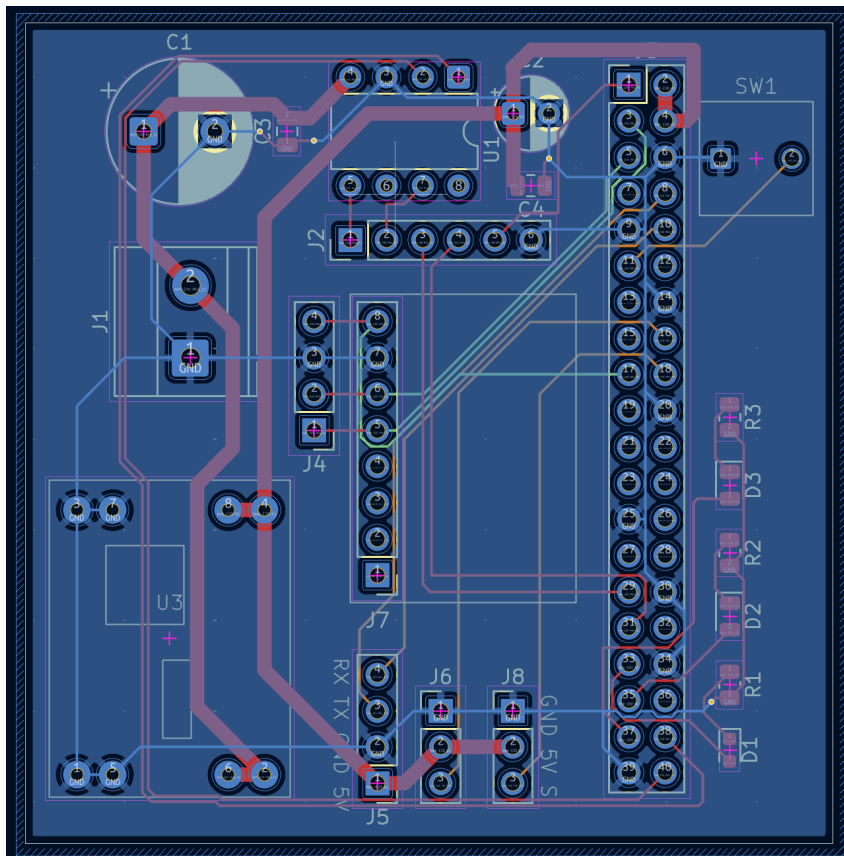


PCB

We also redesigned the PCB such that it fits on the smaller top plate without interfering with the battery holder before sending it out for printing through JLCPCB



We only realised after inserting the PCB onto the raspberrypi that the 2x20 connectors were in the wrong orientation, which almost short circuited the raspberrypi, so we had to redesign the PCB board.



While testing with a new servo, we realised that both servos draw a significant amount of energy, which caused the raspberry pi to reboot. Our solution was to add an extra voltage regulator for the servos to separate the servo and raspberry pi power supply.

